

BOARD OF INTERMEDIATE EDUCATION, KARACHI

Computer Science Class XII Lectures

Units 1 to 7 | Sindh Curriculum 2019

Seven classroom lectures for Computer Science Class XII, written for the Board of Intermediate Education, Karachi.

The lectures follow the Sindh Curriculum for Computer Science, Grades XI-XII, 2019: the system development life cycle, pointers, object-oriented programming, file handling, databases, multimedia, and wireless and mobile communication.

Object-oriented programming carries about a quarter of the Class XII weight. The programs for classes, pointers and files were compiled and run.

Use the same 75-mark paper shape for practice, and confirm the current model paper at biek.edu.pk.

Units inside this file: 1 System development life cycle. 2 Pointers. 3 Object-oriented programming. 4 File handling. 5 Database fundamentals. 6 Introduction to multimedia. 7 Wireless and mobile communication.

Unit	Lecture	Weight
1	System development life cycle	10%
2	Pointers	15%
3	Object-oriented programming in C++	25%
4	File handling	15%
5	Database fundamentals	15%
6	Introduction to multimedia	15%
7	Wireless and mobile communication	10%

Class XII | before the units

How to use the Class XII lectures

Same 75-mark paper shape as Computer Science Paper II

Class XII moves from the computer itself to building software, then to pointers, classes, files, databases, multimedia and wireless communication. The paper shape is the same three-hour, 75-mark pattern described in the Class XI preface: 29 multiple-choice questions, 10 short answers out of 15, and 2 detailed answers out of 3. Confirm the model paper for your year at biek.edu.pk.

What you should be able to do

- Plan revision by the unit weights in the 2019 curriculum.
- Give extra practice to object-oriented programming, which carries the highest weight.

Path through the lecture

- 1 Read Unit 1 as the story of a project. The later programming units are the building and testing phases of that story.
- 2 Type the pointer, class and file programs. They are short and they are examined.
- 3 Work the library database on paper up to third normal form before you write SQL.

Unit weight in Class XII

Unit	Title	Weight	Theory periods	Lab periods
1	System development life cycle	10%	15	-
2	Pointers	15%	5	5
3	Object-oriented programming in C++	25%	30	30
4	File handling	15%	5	5
5	Database fundamentals	15%	20	10
6	Introduction to multimedia	15%	20	20
7	Wireless and mobile communication	10%	15	-

Object-oriented programming is a quarter of the course. Pointers and files are short units with a high weight, so the programs in those units repay careful practice.

Bring it together

- Spend the most programming time on classes, constructors, inheritance and polymorphism.
- Be able to draw an ER diagram and write a SELECT with a WHERE clause.

Class XII | Unit 1 | weight 10%

System development life cycle

Board weight about 10% | 15 theory periods

A system is a set of parts working toward a purpose. Building a software system properly means moving through agreed phases, with a named person responsible for each kind of work.

What you should be able to do

- Define a system and the system development life cycle, and say why the cycle matters.
- Describe the roles of the project manager, analyst, architect, developers, testers, operations and support.
- Explain planning, requirements, design, implementation, testing, deployment and maintenance.
- Distinguish functional and non-functional requirements, and describe an SRS, bugs, errors and an SLA.

Path through the lecture

- 1 Use one system the class knows, such as a college library, and walk it through every phase.
- 2 Put the seven roles on the board beside the phase each role leads.
- 3 Write three functional and three non-functional requirements for that library before defining the SRS.

1.1 Systems and the people who build them

System. A set of interrelated components that work together to achieve a purpose.

System development life cycle. The ordered set of phases through which a software system is planned, built, tested, delivered and kept in service.

The cycle matters because a college that starts coding on the first day often builds the wrong system. Requirements that are written down can be checked. A design can be reviewed before money is spent on programming. Testing finds faults before students meet them. Maintenance keeps the system useful after delivery. The cycle also tells a team who is responsible for each kind of decision.

Stakeholders in the cycle

Role	Leads this work
Project manager	Plan, schedule, cost and coordination of the team
Business analyst	Finding and writing what the organisation needs
System architect	The overall design and structure of the solution
Software developer	Writing the programs
Tester	Checking that the system meets the requirements and reporting faults
Operational team	Putting the system into real use and running it
Production support	Looking after the system after it is live, including repairs and small changes

1.2 The phases



Maintenance feeds new requirements back into planning.

Figure 1.1 The phases used in this course. Maintenance sends new needs back into planning.

Planning and requirements

Problem identification names what is wrong or missing. A library may be losing track of which student has which book. **Planning** sets the scope, the time and the people. **Feasibility** asks whether the system should be built at all.

Feasibility questions

Kind	The question
Technical	Can it be built with the hardware, software and skills available?
Economic	Do the benefits justify the cost?
Operational	Will the librarians and students actually be able to use it?
Schedule	Can it be ready for the term in which it is needed?

Requirement gathering collects what the system must do, from interviews, existing forms and observation. A **functional requirement** is a behaviour: the librarian can issue a book to a member, search by title, and record a return. A **non-functional requirement** is a quality: the search should answer within a few seconds, only a librarian can delete a member, and the screens should be readable on the lab monitors.

Software requirements specification. The document that records what the system must do and the constraints on how well it must do it. Designers and testers work from this document.

An SRS usually contains an introduction, a description of the users, the functional requirements, the non-functional requirements, the data that must be stored, and the interfaces with other systems. It is the agreement between the college and the builders.

Design, build, test, deliver, maintain

Design decides the structure of the software, the data and the screens. An **algorithm** is a finite, ordered, unambiguous set of steps that solves a problem. A **flowchart** is a diagram of those steps. Use an oval for start and stop, a parallelogram for input or output, a rectangle for a process, a diamond for a decision, and arrows for the order.

Implementation is the coding of the design in a language such as C++. **Testing** runs the system to see whether it meets the SRS. A **bug** is a defect in the software. An **error** is a mistake or a wrong result: a person may write a wrong formula (a human error), the program may contain a defect (a bug), and the user may then see a wrong total (a failure or an incorrect result). In answers, define a bug as a fault in the program and an error as a wrong step or a wrong result, and give one example of each.

Deployment puts the finished system into the place where it will be used: the library computers, the real data and the trained staff. **Maintenance** repairs faults, adapts the system when rules change, and improves it. A **service level agreement** is a written commitment between the provider and the customer about the service, for example the hours the system will be available and how quickly a serious fault will be attended to.

Exam point

An 8-mark SDLC answer that only lists five headings will look thin. For each phase write one sentence of work and name the role that leads it. Use the library as the running example.

Key terms

Term	Meaning to write in an answer
System	Interrelated parts working for a purpose.
SDLC	The phases of planning, building, testing, delivering and maintaining software.
Feasibility	A study of whether the proposed system can and should be built.
Functional requirement	A behaviour the system must perform.
Non-functional requirement	A quality or constraint, such as speed or security.
SRS	The specification of requirements that guides design and testing.
Algorithm	A finite sequence of clear steps that solves a problem.
SLA	An agreement on the level of service to be provided.

Bring it together

- The life cycle reduces the chance of building the wrong system.
- Analysts gather requirements, architects design, developers code, testers test, operations deploy, and support maintains.
- Functional requirements say what the system does. Non-functional requirements say how well.
- A bug is a defect in the software. An SLA states the service the live system must provide.

Do this in the practical

- 1 Write six requirements for a college library system: three functional and three non-functional. Draw a flowchart to issue a book.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A functional requirement describes:

- A. What the system does
- B. Only the colour of the case
- C. The brand of keyboard
- D. A topology

2. The SRS is mainly used so that:

- A. Design and test have a written set of requirements
- B. RAM is refreshed
- C. A cable is crimped
- D. A pointer is dereferenced

3. Which role leads the writing of program code?

- A. Software developer
- B. Only the projector
- C. The CMOS battery
- D. The barcode

4. Economic feasibility asks whether:

- A. The benefits justify the cost
- B. The screen is an LCD
- C. The language is C++
- D. The bus is unidirectional

5. A diamond in a flowchart means:

- A. A decision
- B. A start symbol only
- C. A database table
- D. A wireless cell

6. An SLA belongs most naturally to:

- A. Support of a live system
- B. Erasing an EPROM
- C. Declaring an int
- D. A simplex link only

7. Maintenance happens:

- A. After the system is in use, and can lead to new requirements
- B. Only before the problem is identified
- C. Only inside the ALU
- D. Only on paper flowcharts

8. A bug is:

- A. A defect in the software
- B. A network topology
- C. A file attribute
- D. A normal form

Answer key

Q	Answer	Why this is the answer
1	A	Functional requirements are behaviours, such as issuing a book.
2	A	The specification is the agreed description of the system to be built.
3	A	Developers implement the design by coding.
4	A	Economic feasibility compares cost and benefit.
5	A	A diamond is the decision symbol. An oval is start or stop.
6	A	A service level agreement sets the service expected after deployment.
7	A	Maintenance looks after the live system and often starts another cycle.

Q	Answer	Why this is the answer
8	A	A bug is a fault in the program that can cause wrong behaviour.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define the system development life cycle.

Model answer

The system development life cycle is the set of phases through which a system is planned, designed, built, tested, deployed and maintained. It matters because each phase checks the work of the one before it, so the team is less likely to code a system that does not meet the need.

Q2. Differentiate functional and non-functional requirements.

Model answer

A functional requirement states a behaviour, for example that a librarian can record the return of a book. A non-functional requirement states a quality or constraint, for example that the record is saved securely and the screen responds promptly. Both belong in the SRS.

Q3. What is feasibility? Name two kinds.

Model answer

Feasibility is the study of whether a proposed system should be built. Technical feasibility asks whether the tools and skills exist. Economic feasibility asks whether the benefit justifies the cost. Operational and schedule feasibility are the other two kinds used in class.

Q4. Differentiate an algorithm and a flowchart.

Model answer

An algorithm is a written sequence of finite, clear steps that solves a problem. A flowchart is a diagram of those steps, using standard symbols such as a rectangle for a process and a diamond for a decision. The flowchart shows the same logic in a form that is easy to follow on paper.

Q5. What is a service level agreement?

Model answer

A service level agreement is a commitment between a service provider and a customer about the level of service. It can state the hours a system will be available and the time allowed to respond to a serious fault.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the phases of the software development life cycle with the role responsible for each.

- 1 Define system and SDLC, and give one reason the cycle is used.
- 2 Describe problem identification, planning, feasibility and requirements, and name the analyst and the project manager.
- 3 Describe the SRS, then design and the architect, including algorithm and flowchart.
- 4 Describe coding by developers, testing by testers, deployment by operations, and maintenance by production support.
- 5 Define bug and SLA in the last two phases, using a library example throughout.

Question 2. Explain how requirements are gathered and written for a college library system.

- 1 Name the users: librarian, member, administrator.
- 2 Give three functional requirements in complete sentences.
- 3 Give three non-functional requirements: speed, security, usability.
- 4 Say what an SRS contains and who uses it afterwards.
- 5 Mention feasibility briefly so the college does not specify a system it cannot run.

Class XII | Unit 2 | weight 15%

Pointers

Board weight about 15% | 5 theory periods and 5 lab periods

A pointer holds the address of a variable. With that address the program can read or change the variable itself. Pointers are short to write and easy to lose marks on, so the operators and the types have to be exact.

What you should be able to do

- Explain why pointers are used and what a memory address is.
- Use the address-of operator and the dereference operator.
- Declare and initialise a pointer.
- Write a program that displays an address and a value, and a function that changes variables through pointers.

Path through the lecture

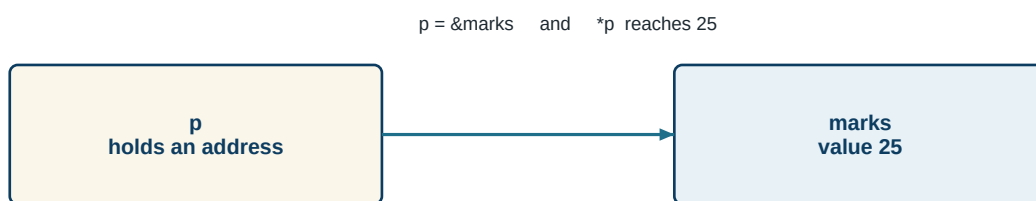
- 1 Draw a variable as a box and a pointer as a box that contains the address of the first box.
- 2 Type the single-variable program and read both the address and the value.
- 3 Pass two addresses into a function and show that the originals change.

2.1 Addresses and pointer variables

Pointer. A variable whose value is the address of another variable.

Every variable occupies memory, and that place has an address. The **address-of operator** & produces the address of a variable. The **dereference operator** * uses an address to reach the variable that lives there. Declaring `int *p;` means p is a pointer to an int. It does not mean that p is an int.

Initialising the pointer matters. `int *p = &marks;` stores the address of marks in p. After that, *p and marks name the same value. Changing *p changes marks. A pointer that has never been set must not be dereferenced. It does not yet point at a variable you own.



The pointer stores where the variable lives, not a second copy of the value.

Figure 2.1 p stores the address of marks. *p is the value 25.

Pointers matter because a function can receive an address and change the caller's variable, because dynamic structures such as linked data are built from addresses, and because a large value can be shared by passing its address rather than copying it. In this unit the required skill is the simple use: take an address, store it, and reach the value.

Program 2.1 Address and value through a pointer

```
#include <iostream>
using namespace std;

int main() {
    int marks = 25;
    int *p = &marks;
    cout << "Value = " << marks << endl;
    cout << "Address stored in p = " << p << endl;
    cout << "Through pointer = " << *p << endl;
    *p = 40;
    cout << "New value = " << marks << endl;
    return 0;
}
```

The address printed for p changes from run to run. That is expected. The value 25, and the new value 40 after `*p = 40`, stay the same. The program shows both operators: `&marks` goes into p, and `*p` comes back to the value.

2.2 Pointers as function arguments

A function parameter of type `int *` receives an address. Inside the function, `*a` reaches the original variable. At the call, pass `&first`, not `first`. Passing `first` would pass the number, and the types would not match.

Program 2.2 Double two integers through pointers

```
#include <iostream>
using namespace std;

void doubleBoth(int *a, int *b) {
    *a = *a * 2;
    *b = *b * 2;
}

int main() {
    int first, second;
    cout << "Enter two integers: ";
    cin >> first >> second;
    doubleBoth(&first, &second);
    cout << "First = " << first << endl;
    cout << "Second = " << second << endl;
    return 0;
}
```

Input 4 and 7 prints 8 and 14. This is the pointer form of pass by reference. The `&` form you used in Class XI (`int &n`) and the pointer form (`int *n` plus `*n`) both let a function change the original. The pointer form makes the address visible, which is the point of this unit.

Exam point

In an exam, the declaration `int *p`; the assignment `p = &x`; and the use `*p` are three separate marks. Write all three. State the type the pointer points to.

Key terms

Term	Meaning to write in an answer
Address	The location of a variable in memory.
Pointer	A variable that stores an address.
&	The address-of operator. <code>&x</code> is the address of x.
*	The dereference operator. <code>*p</code> is the object p points to.

Term	Meaning to write in an answer
NULL	A pointer value that points nowhere. Do not dereference it.

Bring it together

- A pointer stores an address. & takes an address. * follows an address.
- The declaration `int *p` says that *p is an int, so p points to an int.
- A function changes a caller's variable when it receives the variable's address and assigns through *.

Do this in the practical

- 1 Display the address and the value of a variable using a pointer.
- 2 Pass the addresses of two variables to a function and double both values.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. The operator & produces:

- A. The address of its operand
- B. The square of a number
- C. A new class
- D. A file mode

2. If p points to marks, then *p is:

- A. The value of marks
- B. The name of the file
- C. Always zero
- D. The layer number

3. `int *p;` declares:

- A. A pointer to int
- B. An int that cannot change
- C. A string
- D. A table

4. Which call matches the parameter `int *a`?

- A. `doubleBoth(&first, &second)`
- B. `doubleBoth(first, second)`
- C. `doubleBoth(*first, *second)`
- D. `doubleBoth(a, b, c)`

5. Assigning `*p = 40;` when p points to marks:

- A. Changes marks to 40
- B. Deletes the program
- C. Changes the topology
- D. Closes a file

6. A pointer should be initialised so that:

- A. It holds the address of a real variable before it is dereferenced
- B. It holds a class name
- C. It replaces main
- D. It stores a flowchart

7. Passing a pointer into a function is useful because:

- A. The function can change the original variable
- B. The keyboard becomes wireless
- C. ROM becomes volatile
- D. Arrays become illegal

8. The value printed as the address of a local variable:

- A. Can differ from one run to the next
- B. Is always 25
- C. Is the source code
- D. Is the SRS

Answer key

Q	Answer	Why this is the answer
1	A	& is the address-of operator.
2	A	Dereferencing p reaches the variable it points to.
3	A	The * in the declaration belongs with p and makes p a pointer.
4	A	The function expects addresses, and &first is an address.
5	A	*p names the same memory as marks.
6	A	Dereferencing an unset pointer does not refer to a known variable.
7	A	The function receives the address and writes through it.
8	A	The system chooses where to place the variable each run.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define a pointer and state the purpose of & and *.

Model answer

A pointer is a variable that stores the address of another variable. The operator & gives the address of a variable. The operator * uses a pointer to reach the variable at that address. After `int *p = &marks`, *p and marks refer to the same value.

Q2. Write the declaration and initialisation of a pointer to an int variable named marks.

Model answer

`int marks = 25; int *p = &marks;` The declaration `int *p` says p points to an int. The initialiser stores the address of marks. *p then reads or changes marks.

Q3. Why is a pointer parameter able to change the caller's variable?

Model answer

The caller passes the address, for example &first. The parameter holds that address. An assignment to *a writes into the caller's variable, not into a copy of the number.

Q4. Give two uses of pointers.

Model answer

A function can modify the variables of its caller by receiving their addresses. Programs also use addresses to build data structures whose size can grow, and to share a large object without copying it.

Q5. What is wrong with dereferencing a pointer that has not been initialised?

Model answer

The pointer does not yet hold the address of a variable the program owns. Using * on it reaches an unknown location. Initialise the pointer with &variable, or set it only from a known object, before you dereference it.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain pointers with a program that shows the address and the value of a variable.

- 1 Define pointer, address, & and *.
- 2 Declare an int and a pointer, and initialise the pointer with the address of the int.
- 3 Print the variable, the pointer, and the dereferenced pointer.
- 4 Change the variable through *p and print the variable again to show it changed.
- 5 State one reason this is useful in a function call.

Question 2. Write a function that receives the addresses of two integers and doubles both. Explain each operator at the call and inside the function.

- 1 Write the function heading with two int* parameters.
- 2 Inside, assign *a = *a * 2 and the same for b.
- 3 In main, read two integers and call the function with &first and &second.
- 4 Explain why the arguments need & and why the body needs *.
- 5 Give the sample 4 and 7 becoming 8 and 14.

Class XII | Unit 3 | weight 25%

Object-oriented programming in C++

Board weight about 25% | 30 theory periods and 30 lab periods

This unit carries a quarter of Class XII. A class describes a kind of object. An object is one real instance. Encapsulation, inheritance and polymorphism are the three ideas the paper expects you to define, to distinguish, and to show in code.

What you should be able to do

- Explain encapsulation, inheritance and polymorphism, and the relationship of class and object.
- Use private and public members, and protected when a derived class must see them.
- Write constructors, including overloading, and a destructor.
- Write a derived class that uses members of its base class.
- Overload member functions and override a base function in a derived class.

Path through the lecture

- 1 Build the Student class on the first day: private data, public functions, one object.
- 2 Add a default constructor and a parameterised constructor with the Time class.
- 3 Inherit DriveClass from Base and show protected data being used.
- 4 Finish with one class that overloads area, and a derived class that overrides show.

3.1 Classes, objects and the three principles

Object-oriented programming. A way of organising a program as objects. An object combines data and the functions that work on that data. Classes describe those objects.

Class. A user-defined type that specifies the data members and member functions common to a kind of object.

Object. One instance of a class. It has its own copies of the data members and it can call the member functions.

Encapsulation places data and the functions that use it inside a class, and controls who may touch the data. **Inheritance** lets a new class receive members from an existing class, so common code is written once. **Polymorphism** lets one name mean more than one function, chosen from the arguments or from the kind of object.

A class has **data members**, the values stored in each object, and **member functions**, the operations. Data members are usually private. Member functions that the rest of the program may call are public. The public functions are the interface. The private data is the detail that can change without rewriting every caller.

3.2 Encapsulation and access

Access specifiers

Specifier	Who can use the member
private	Member functions of the same class. This is the default if you write nothing.
public	Any code that has the object.
protected	The class itself and classes derived from it. Not the general caller.

Data hiding. Keeping data members private so that outside code cannot change them directly. Outside code calls a public function, and that function can check the value.

A **constructor** is a member function with the same name as the class and no return type. It runs automatically when an object is created, so the object starts in a known state. A **default constructor** takes no arguments. A **parameterised constructor** takes values and copies them into the data members. **Constructor overloading** means providing more than one constructor, with different parameter lists. A **destructor** has the class name with a tilde in front, such as `~Time()`, takes no arguments, and runs when the object is destroyed. Use it to release anything the object acquired. In the simple classes in this unit the destructor is often empty, but you should be able to write one.

Program 3.1 Class Student with private data and public functions

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;
    float percentage;
public:
    void setData(int a, float p) {
        age = a;
        percentage = p;
    }
    void showData() {
        cout << "Age: " << age << endl;
        cout << "Percentage: " << percentage << endl;
    }
};

int main() {
    Student s;
    int age;
    float percentage;
    cout << "Enter age and percentage: ";
    cin >> age >> percentage;
    s.setData(age, percentage);
    s.showData();
    return 0;
}
```

Main cannot write `s.age = 17` if age is private. It calls `setData`, and `showData` prints the values. That is encapsulation. Input 17 and 88 prints those values through the object, not through the variables in main.

Program 3.2 Default and parameterised constructors

```
#include <iostream>
using namespace std;

class Student {
private:
    int age;
    float percentage;
public:
    Student() {
        age = 0;
        percentage = 0;
    }
    Student(int a, float p) {
        age = a;
        percentage = p;
    }
    void showData() {
        cout << "Age: " << age << endl;
        cout << "Percentage: " << percentage << endl;
    }
};

int main() {
    Student s(17, 91);
    s.showData();
    return 0;
}
```

Program 3.3 Class Time with two constructors

```
#include <iostream>
using namespace std;

class Time {
private:
    int hours;
    int minutes;
    int seconds;
public:
    Time() {
        hours = 0;
        minutes = 0;
        seconds = 0;
    }
    Time(int h, int m, int s) {
        hours = h;
        minutes = m;
        seconds = s;
    }
    void show() {
        cout << "Time = " << hours << ":" << minutes
            << ":" << seconds << endl;
    }
};

int main() {
    int h, m, s;
    cout << "Enter hours, minutes and seconds: ";
    cin >> h >> m >> s;
    Time t(h, m, s);
    t.show();
    Time empty;
    empty.show();
    return 0;
}
```

Time t(h, m, s) calls the constructor with three arguments. Time empty; calls the default constructor and prints 0:0:0. The two constructors are an overload. There is no return type in front of Time.

3.3 Inheritance

Inheritance. A mechanism by which a derived class receives members from a base class and can add its own members.

Reusability is the practical gain. The base class holds the common members. Each derived class adds what is special to it, instead of copying the common code. The **base class** is the existing class. The **derived class** is the new class, written class DriveClass : public Base. Public inheritance means the public members of the base stay public for users of the derived class.

Protected members are the ones a derived class may use while the rest of the program still cannot. In the program below, name and age are protected in Base, so DriveClass can rely on the base functions that set them. Roll belongs only to the derived class.

Program 3.4 A derived class using a base class

```
#include <iostream>
#include <string>
using namespace std;

class Base {
protected:
    string name;
    int age;
public:
    void setBase(string n, int a) {
        name = n;
        age = a;
    }
    void showBase() {
        cout << "Name: " << name << endl;
        cout << "Age: " << age << endl;
    }
};

class DriveClass : public Base {
private:
    int roll;
public:
    void setDrive(string n, int a, int r) {
        setBase(n, a);
        roll = r;
    }
    void showDrive() {
        showBase();
        cout << "Roll: " << roll << endl;
    }
};

int main() {
    DriveClass student;
    string name;
    int age, roll;
    cout << "Enter name, age and roll: ";
    cin >> name >> age >> roll;
    student.setDrive(name, age, roll);
    student.showDrive();
    return 0;
}
```

The object in main is a DriveClass, not a Base. setDrive calls setBase, so the derived class reuses the base behaviour. The printed name, age and roll show both parts of the object.

3.4 Polymorphism

Polymorphism. One interface, more than one function. In this course that means overloading and overriding.

Overloading a member function means several functions in the same class share a name and differ in their parameter lists. `area(int)` and `area(int, int)` can both exist. The call decides which one runs. This is the same rule as function overloading in Class XI.

Overriding means a derived class provides its own version of a function that the base class already has. The derived version replaces the base version for a derived object. If the base function is marked `virtual` and the call is made through a base pointer that actually points to a derived object, the derived function runs. That is the form of overriding that behaves according to the real object. A function that is not virtual and is called through a base pointer would run the base version, so the examples in this lecture mark show as virtual.

Program 3.5 Overloading area and overriding show

```
#include <iostream>
using namespace std;

class Base {
public:
    void area(int side) {
        cout << "Square = " << side * side << endl;
    }
    void area(int length, int width) {
        cout << "Rectangle = " << length * width << endl;
    }
    virtual void show() {
        cout << "Base show" << endl;
    }
};

class Drive : public Base {
public:
    void show() override {
        cout << "Drive show" << endl;
    }
};

int main() {
    Base shape;
    shape.area(4);
    shape.area(4, 5);
    Drive d;
    Base *p = &d;
    p->show();
    return 0;
}
```

`shape.area(4)` prints 16. `shape.area(4, 5)` prints 20. The pointer `p` has type `Base*` and points at a `Drive` object. Because `show` is virtual, the call prints `Drive show`. If you remove `virtual`, the same call prints `Base show`. That single experiment is the best way to remember why the keyword is there.

Overloading and overriding

Basis	Overloading	Overriding
Where	Same class	Derived class and its base
Names	Same name	Same name
Parameter list	Must differ	Stays the same
What chooses the function	The arguments at the call	The kind of object, when the function is virtual

Exam point

A full-mark class program has: the class heading, private data, a public constructor or setter, a public display function, an object in main, a call, and matching braces. Inheritance adds the colon, public Base, and at least one use of a base member.

Key terms

Term	Meaning to write in an answer
Class	The description of data members and member functions.
Object	One instance of a class.
Encapsulation	Data and functions bound together, with access controlled.
Data hiding	Private data changed only through member functions.
Constructor	A special function that initialises an object when it is created.
Destructor	A special function that runs when an object is destroyed.
Base class	The class that is inherited from.
Derived class	The class that inherits.
Overriding	A derived class replacing a base function of the same signature.

Bring it together

- A class is a type. An object is one value of that type.
- Private data and public functions are encapsulation. Protected data is visible to derived classes.
- Constructors initialise. You may overload them. A destructor tidies up.
- Inheritance reuses a base class. Polymorphism in this unit is overloading plus overriding.

Do this in the practical

- 1 Define Student with age and percentage. Input the values and show them through an object.
- 2 Initialise a Student and a Time object with constructors.
- 3 Inherit a derived class from a base class and show data through the derived object.
- 4 Demonstrate overloading of area and overriding of show.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. An object is:

- A. An instance of a class
- B. A transmission medium
- C. A file mode
- D. A normal form

2. Private members can be used directly by:

- A. Member functions of the same class
- B. Every function in the program
- C. Only the operating system
- D. Only SQL

3. A constructor:

- A. Has the class name and no return type
- B. Is named main
- C. Must return int
- D. Is a topology

4. Public inheritance is written:

- A. class Drive : public Base
- B. class Drive inherits Base
- C. Drive extends public
- D. public(Drive, Base)

5. Protected members are available to:

- A. The class and its derived classes
- B. Every file on the disk
- C. Only the linker, never a derived class
- D. Only constructors of unrelated classes

6. Overloading is distinguished by:

- A. The parameter list
- B. The return type alone
- C. The file name
- D. The colour of the IDE

7. A virtual function lets a base pointer call:

- A. The derived override when the object is derived
- B. Only ROM
- C. A flowchart
- D. The power supply

8. The destructor name for class Time is:

- A. ~Time
- B. delete Time
- C. Time()
- D. end Time

Answer key

Q	Answer	Why this is the answer
1	A	The class describes the kind. The object is one instance.
2	A	Private means the rest of the program must go through the class's own functions.
3	A	Constructors are named after the class and do not have a return type.
4	A	The colon and public Base are the C++ syntax.
5	A	Protected sits between private and public for inheritance.
6	A	Overloaded functions share a name and differ in their parameters.
7	A	virtual selects the function from the actual object.

Q	Answer	Why this is the answer
8	A	The destructor is a tilde followed by the class name.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Differentiate a class and an object.

Model answer

A class is a user-defined type that describes data members and member functions. An object is one instance of that class and holds its own data. Student is the class. One pupil, s, is an object. Many objects can be created from one class.

Q2. Define encapsulation and data hiding.

Model answer

Encapsulation binds data and the functions that operate on it into a class, and uses access specifiers to control access. Data hiding keeps the data private so outside code cannot assign to it directly. A public function can check a value before storing it.

Q3. Differentiate a constructor and a destructor.

Model answer

A constructor runs when an object is created and initialises it. It has the class name and no return type. A destructor runs when the object is destroyed and releases what the object acquired. Its name is ~ followed by the class name. A class may overload constructors. There is one destructor.

Q4. What does public inheritance give the derived class?

Model answer

The derived class receives the members of the base and can add its own. Public members of the base remain available to users of the derived object. Protected members of the base can be used inside the derived class. Private members of the base stay accessible only through the base class's own functions.

Q5. Differentiate overloading and overriding.

Model answer

Overloading defines several functions of the same name in one class with different parameter lists. The arguments select the function. Overriding defines, in a derived class, a function with the same signature as a base function. For a derived object the derived function replaces the base one. Mark the base function virtual when the call is made through a base pointer.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain encapsulation, inheritance and polymorphism, and illustrate each with C++.

- 1 Define OOP, class and object.
- 2 Explain encapsulation with private data and a public set and show function. Mention the access specifiers.
- 3 Explain inheritance with base, derived, public inheritance and reusability. Mention protected.
- 4 Explain polymorphism as overloading and overriding, with the virtual keyword for a real override through a pointer.
- 5 Use one small sketch of code for each idea rather than one unrelated program.

Question 2. Explain constructors, constructor overloading and the creation of objects. Use the class Time.

- 1 Define a constructor and state that it has no return type.
- 2 Write a default constructor that stores zeroes and a constructor that copies hours, minutes and seconds.
- 3 Show Time empty; and Time t(h, m, s); and say which constructor each one calls.
- 4 Show a display function.
- 5 Add one sentence on the destructor and when it runs.

Class XII | Unit 4 | weight 15%

File handling

Board weight about 15% | 5 theory periods and 5 lab periods

Variables disappear when the program ends. A file keeps data on secondary storage. This lecture opens files, chooses a mode, and reads and writes both text and binary data.

What you should be able to do

- Define file handling and distinguish text and binary files.
- Open a file for reading, writing or appending.
- Explain BOF and EOF, and single-character, string and binary streams.
- Write programs that create and read a text file and a binary file of student records.

Path through the lecture

- 1 Show a text file in an editor, then show that a binary file is not meant to be read that way.
- 2 Teach open, check, use, close as a fixed four-step habit.
- 3 Write the student text program, then the same student as a binary record.

4.1 Files, modes and ends

File handling. The part of a program that creates, writes, reads and updates data stored in a file on secondary storage.

A **text file** stores characters. You can open it in an editor and read the numbers and words. A **binary file** stores the bytes of the data as the program holds them in memory. It is compact and it preserves the exact bits of an int or a float. It is not a document for a person to edit.

Modes used with ifstream, ofstream and fstream

Mode	Effect
ios::in	Open for reading. The file should already exist.
ios::out	Open for writing. A new file is created. An existing file is emptied first.
ios::app	Open for appending. New output is added at the end. Existing contents stay.
ios::binary	Treat the file as binary. Combine it with in or out, for example ios::in ios::binary.

ofstream is for output, ifstream is for input, and fstream can do both. After open, test the stream before you use it. Close the file when the work is finished so the buffer is written and the file is released.

BOF. Beginning of file. The position before the first byte, where a newly opened input file starts.

EOF. End of file. The condition that becomes true when a read goes past the last data. The loop while (inFile >> word) stops when no more input can be read.

A stream is a flow of data. **Single-character I/O** uses get and put. **String I/O** uses >>, getline and <<. **Binary I/O** uses read and write with a memory address and a size. write(reinterpret_cast<char*>(&s), sizeof(s)) copies the bytes of the object s into the file. read copies them back into an object of the same type.

4.2 Text and binary programs

Program 4.1 Write and read a text file

```

#include <iostream>
#include <fstream>
#include <string>
using namespace std;

int main() {
    ofstream outFile("student.txt");
    outFile << "Ayesha" << endl;
    outFile << 88 << endl;
    outFile.close();

    ifstream inFile("student.txt");
    string name;
    int marks;
    inFile >> name >> marks;
    cout << "Read: " << name << endl;
    cout << "Read: " << marks << endl;
    inFile.close();
    return 0;
}

```

Program 4.1 creates student.txt, writes a name and a mark, closes it, then opens it again and reads both values. The file remains on the disk after the program ends. That is the whole point of file handling.

Program 4.2 Write and read a binary student record

```

#include <iostream>
#include <fstream>
#include <cstring>
using namespace std;

struct Student {
    char name[20];
    int age;
    float percentage;
};

int main() {
    Student s;
    strcpy(s.name, "Ayesha");
    s.age = 17;
    s.percentage = 88.5f;

    ofstream outFile("student.bin", ios::binary);
    outFile.write(reinterpret_cast<char*>(&s), sizeof(s));
    outFile.close();

    Student loaded;
    ifstream inFile("student.bin", ios::binary);
    inFile.read(reinterpret_cast<char*>(&loaded), sizeof(loaded));
    inFile.close();

    cout << "Name: " << loaded.name << endl;
    cout << "Age: " << loaded.age << endl;
    cout << "Percentage: " << loaded.percentage << endl;
    return 0;
}

```

The student is a structure with a char array, an int and a float, so the object can be copied as a block of bytes. The name is placed with strcpy so the array contains a proper C string. Reading uses the same struct and the same size. If the reading program used a different layout, the bytes would be interpreted wrongly.

Exam point

State text versus binary before you write code in a long question. Then show open, the transfer, close, and a second open to read the data back. Mention EOF if the question asks how a loop knows when to stop.

Key terms

Term	Meaning to write in an answer
Text file	A file stored as characters.
Binary file	A file stored as the raw bytes of the data.
ofstream	A stream used to write a file.
ifstream	A stream used to read a file.
Append	Adding new data at the end of an existing file.
EOF	The end-of-file condition, true when no more data can be read.

Bring it together

- Files keep data after the program ends.
- Text files store characters. Binary files store memory images of objects.
- Choose in, out or app, and add binary when the bytes must be preserved.
- Open, check, read or write, and close. EOF tells an input loop to stop.

Do this in the practical

- 1 Write a name and a mark to a text file and read them back.
- 2 Write a student structure to a binary file and read it into a second variable.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A text file stores:

- A. Characters
- B. Only electrical addresses
- C. Flowchart symbols
- D. OSI layers

2. ios::app means:

- A. Append at the end of the file
- B. Erase ROM
- C. Sort an array
- D. Close the monitor

3. ios::out on an existing file:

- A. Opens it for writing and discards the old contents
- B. Always appends
- C. Converts it to a pointer
- D. Marks it read-only in Windows

4. EOF becomes true when:

- A. A read passes the last data
- B. The computer boots
- C. A class is declared
- D. Wi-Fi connects

5. Binary read and write transfer:

- A. A block of bytes of a given size
- B. Only one flowchart
- C. Only comments
- D. A topology name

6. Which header declares file streams?

- A. fstream
- B. iostream only, with no file support at all
- C. cmath
- D. cstring only

7. Closing a file:

- A. Flushes the buffer and releases the file
- B. Deletes the compiler
- C. Changes the address bus
- D. Is illegal in C++

8. A binary student record should be read into:

- A. An object with the same member layout
- B. A different unrelated type on purpose
- C. An OSI layer
- D. A service level agreement

Answer key

Q	Answer	Why this is the answer
1	A	Text files hold character data that an editor can show.
2	A	Append keeps existing data and adds at the end.
3	A	Output mode creates or empties the file. Append mode keeps it.
4	A	End of file means there is nothing left to read.
5	A	read and write copy a stated number of bytes.
6	A	File stream classes are declared in fstream. iostream covers cin and cout.
7	A	close completes pending output and frees the file.
8	A	The bytes match the struct that was written.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Differentiate a text file and a binary file.

Model answer

A text file stores data as characters and can be opened in an editor. A binary file stores the bytes of values as they are held in memory, using read and write. Binary storage preserves exact numeric bits and is not meant to be edited as a document.

Q2. Differentiate write mode and append mode.

Model answer

ios::out opens a file for writing and empties it if it already exists. ios::app opens a file so that new output is added after the existing contents. Use out when the file should be created fresh. Use app when older records must remain.

Q3. What are BOF and EOF?

Model answer

BOF is the beginning of the file, the position where reading normally starts. EOF is the end-of-file condition, which becomes true when a read can obtain no more data. An input loop stops when EOF is reached.

Q4. Write the statements that open a text file for output and write a name.

Model answer

ofstream outFile("student.txt"); outFile << name << endl; outFile.close(); The first statement creates or empties the file. The second writes the name. The third closes it.

Q5. Why must the struct used to read a binary record match the struct that wrote it?

Model answer

write copies the bytes of the object in memory order. read copies those same bytes back. A different member order or a different size would place the bytes into the wrong members, so the name, age or percentage would be wrong.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain file modes and the difference between text and binary handling. Include BOF and EOF.

- 1 Define file handling and why files outlive variables.
- 2 Contrast text and binary with one example of each transfer: << and >>, then read and write.
- 3 Explain in, out, app and binary.
- 4 Define BOF and EOF and show them in a read loop.
- 5 Mention open, test, use and close as the safe order.

Question 2. Describe a program that stores a student in a binary file and reads the student back.

- 1 Define a struct with a char name, an int age and a float percentage.
- 2 Fill the fields, using strcpy for the name.
- 3 Open an ofstream with ios::binary and write sizeof bytes.
- 4 Close, open an ifstream with ios::binary, and read into a second struct.
- 5 Print the fields to show the data survived.

Class XII | Unit 5 | weight 15%

Database fundamentals

Board weight about 15% | 20 theory periods and 10 lab periods

A database stores related data so that many programs can use it without keeping their own conflicting copies. This lecture moves from that idea to tables, relationships, normal forms and SQL, and ends with the design of a library.

What you should be able to do

- Contrast a flat file with a database management system and name DBMS components, characteristics, models and users.
- Describe the three levels of data abstraction and the basic relational terms.
- Draw entities, attributes and relationships, and apply 1NF, 2NF and 3NF.
- Use keys, integrity constraints and the four relationship types.
- Write SQL for selection, projection, conditions, joins and simple DML.
- Design a library schema through the standard design steps.

Path through the lecture

- 1 Start with one messy spreadsheet of library issues and show the repeated names.
- 2 Normalise that sheet to 3NF on the board. The SQL afterwards uses those tables.
- 3 Keep selection and projection in one pair of examples so the words stay attached to WHERE and to the column list.

5.1 From files to a database

A **flat file** is a single file of records, often one fact repeated on many rows, with no enforced link to other files. A class list in one spreadsheet and an issue list that copies every student's name into every row is a flat-file habit. A **database management system** is software that stores related data, controls access, and lets programs ask questions without each program knowing the physical layout.

Databases run college records, bank accounts, hospital patients, shop stock and examination results. Anywhere the same fact is used by more than one process, a shared database avoids a pile of disagreeing copies.

Components of a DBMS

Component	Meaning
Hardware	The computers and disks the data lives on.
Software	The DBMS itself, together with the application programs.
Data	The stored facts, and the data dictionary that describes them.
Procedures	The rules for backup, access and recovery.
Data access language	The language used to define and manipulate the data. SQL is the one in this course.

Characteristics you should be able to name

Characteristic	What it means here
Controlled redundancy	A fact is stored once and referred to, rather than copied everywhere.
Data accuracy	The stored values match the real world they describe.

Characteristic	What it means here
Integrity	The data obeys rules, such as a mark being within range and a book issue pointing at a real book.
Normalisation	A design discipline that splits tables so that redundancy and update problems shrink.
Security	Only authorised users perform the actions allowed to them.
Scalability	The system can grow in data and users.

The **entity-relationship model** describes the world as entities, attributes and relationships, before tables are chosen. The curriculum calls this the entity relational model. The **relational model** stores data in tables. An **object-relational model** keeps tables and adds object features such as richer data types. Draw the ER model when the question says conceptual design. Draw tables when it says relational.

A **database administrator** controls the structure, the accounts, the backup and the security. A **database application programmer** writes the programs that use the database, such as the library screens.

5.2 Architecture, tables and normal forms

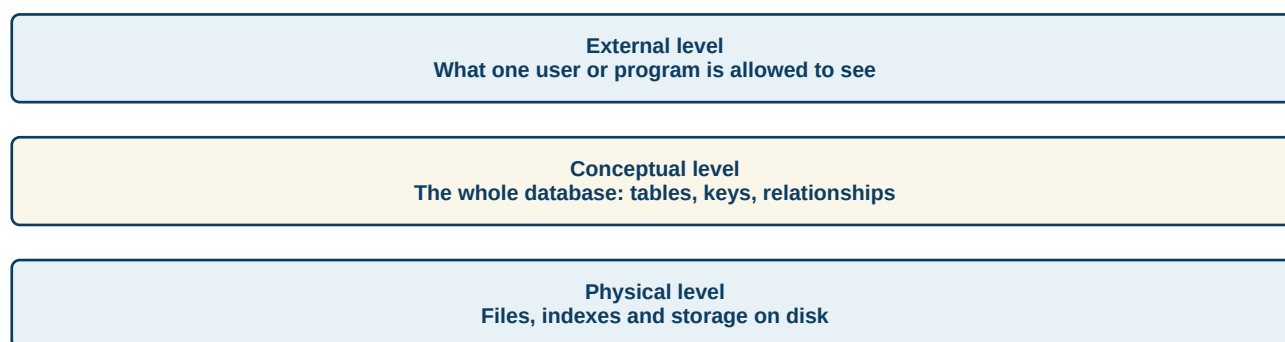


Figure 5.1 Three levels of data abstraction.

The **external level** is a view for one kind of user. A student may see his own issues and not every member's phone number. The **conceptual level** is the whole logical database: all the tables, keys and relationships. The **physical level** is how the data is stored in files and indexes. Users work at the top. The DBMS hides the bottom. This separation is data abstraction.

Relational words

Word	Meaning
Table	A named collection of rows of the same kind. Also called a relation.
Record or tuple	One row.
Field, column or attribute	One named property, such as Title.
Schema	The design of the tables, columns and keys, as distinct from the rows currently stored.
Primary key	A column, or a group of columns, that identifies one row and never repeats.
Foreign key	A column that refers to the primary key of another table.

In an ER diagram an **entity** is a rectangle, an **attribute** is an oval joined to its entity, and a **relationship** is a diamond joined to the entities it connects. Underline the key attribute. For the library, BOOK, MEMBER and the relationship ISSUES are enough to start.

Relationship types

Type	Library meaning
One to one	A member has one current library card. That card belongs to one member.
One to many	One member may borrow many books. From the book side, one copy is issued to one member at a time.
Many to one	Many issue-rows point at one member. This is the same fact as one-to-many, read from the other side.
Many to many	Over time many members borrow many books. A junction table, ISSUE, holds one row per loan and breaks the many-to-many into two one-to-many links.

A normalisation you can reproduce

Start with this repeated sheet. The intended key of one result row is StudentID together with SubjectID.

Before the split

StudentID	SubjectID	StudentName	Dept	HOD	SubjectName	Marks
S1	M	Ayesha	Science	Khan	Maths	80
S1	P	Ayesha	Science	Khan	Physics	74
S2	M	Bilal	Science	Khan	Maths	61

First normal form requires atomic values and no repeating group. Each cell above already holds one value, and each subject is its own row, so the sheet is in 1NF. A cell that contained 'Maths, Physics' would have to be split into two rows first.

Second normal form requires 1NF, and every non-key column must depend on the whole key, not on part of it. StudentName depends only on StudentID. SubjectName depends only on SubjectID. Those are partial dependencies. Move them out. Marks depends on the whole key, so it stays.

After 2NF

Table	Columns	Key
STUDENT	StudentID, StudentName, Dept, HOD	StudentID
SUBJECT	SubjectID, SubjectName	SubjectID
RESULT	StudentID, SubjectID, Marks	StudentID + SubjectID

Third normal form requires 2NF, and no non-key column may depend on another non-key column. In STUDENT, HOD depends on Dept, and Dept depends on StudentID. That is a transitive dependency. Move the department out. The curriculum's heading once writes 3NF beside the letters 2NF. Write **3NF** and mean third normal form.

After 3NF

Table	Columns	Key
DEPT	Dept, HOD	Dept
STUDENT	StudentID, StudentName, Dept	StudentID, and Dept refers to DEPT
SUBJECT	SubjectID, SubjectName	SubjectID
RESULT	StudentID, SubjectID, Marks	StudentID + SubjectID

Changing the head of Science now changes one row in DEPT, not every student row. That is the practical gain.

Integrity constraints protect that design. A **NOT NULL** constraint refuses an empty student name. A **primary key** refuses a duplicate StudentID and refuses to leave it empty. A **foreign key** refuses a RESULT row whose StudentID is not in STUDENT, and refuses a STUDENT whose Dept is not in DEPT.

5.3 SQL

SQL. Structured Query Language. The language used to define tables and to retrieve and change the data in a relational database.

Projection chooses columns. **Selection** chooses rows. In SQL the column list after SELECT is the projection, and the WHERE clause is the selection.

Projection and selection

```
SELECT StudentName, Dept
FROM STUDENT;
```

```
SELECT StudentName, Dept
FROM STUDENT
WHERE Dept = 'Science';
```

The first statement projects two columns of every row. The second also selects the Science rows. Arithmetic in the select list calculates a value: SELECT Marks, Marks + 5 FROM RESULT. Comparison operators are =, <>, >, <, >= and <=. Logical operators are AND, OR and NOT.

An **equijoin** combines rows that have equal key values. A **cross join** combines every row of one table with every row of another and is rarely the result you want. The explicit join and the older WHERE form both appear in classrooms. Know one of them fluently and recognise the other.

Equijoin of students with their marks

```
SELECT STUDENT.StudentName, RESULT.SubjectID, RESULT.Marks
FROM STUDENT
INNER JOIN RESULT
ON STUDENT.StudentID = RESULT.StudentID
WHERE RESULT.Marks >= 50;
```

Create, insert, update and delete

```
CREATE TABLE STUDENT (
    StudentID VARCHAR(6) PRIMARY KEY,
    StudentName VARCHAR(40) NOT NULL,
    Dept VARCHAR(20)
);

INSERT INTO STUDENT (StudentID, StudentName, Dept)
VALUES ('S1', 'Ayesha', 'Science');

UPDATE STUDENT
SET Dept = 'Arts'
WHERE StudentID = 'S1';

DELETE FROM STUDENT
WHERE StudentID = 'S1';

DROP TABLE STUDENT;
```

Microsoft Access uses the same SELECT, WHERE, INSERT, UPDATE and DELETE ideas. In Access query criteria the wildcard for a pattern is often *, while standard SQL uses % with LIKE. Use the form your lab software accepts, and name the operation correctly in the written paper.

5.4 Designing the library

- 1 Purpose: record books, members and which member has which book.
- 2 Tables: BOOK, MEMBER and ISSUE. ISSUE is needed because borrowing is many-to-many over time.

- 3 Fields: BOOK has BookID, Title, Author and Subject. MEMBER has MemberID, MemberName and Class. ISSUE has IssueID, BookID, MemberID, IssueDate and ReturnDate.
- 4 Unique fields: BookID, MemberID and IssueID are the primary keys.
- 5 Relationships: ISSUE.BookID refers to BOOK. ISSUE.MemberID refers to MEMBER. Each relationship is many issues to one book or one member, and the design is in 3NF because names and titles are not repeated on every loan.
- 6 Refine: a ReturnDate may be empty while the book is out, so that column allows null. BookID on a loan must not be null.

Exam point

If the long question is the library, draw three rectangles, two relationships, the primary keys underlined, and the foreign keys marked FK. Then write one SELECT that lists the member name and the book title for loans not yet returned.

Key terms

Term	Meaning to write in an answer
DBMS	Software that manages a shared, structured collection of related data.
Schema	The design of the database.
Primary key	The column or columns that identify one row.
Foreign key	A column that refers to a primary key in another table.
1NF	Atomic values, with no repeating group.
2NF	1NF, and no partial dependency on a composite key.
3NF	2NF, and no transitive dependency.
Selection	Choosing rows, written with WHERE.
Projection	Choosing columns, written in the SELECT list.

Bring it together

- A DBMS shares related data with control of redundancy, integrity and security.
- External, conceptual and physical levels separate the user's view from storage.
- ER diagrams use rectangles, ovals and diamonds. Tables then receive keys.
- 1NF, 2NF and 3NF remove repeating groups, partial dependencies and transitive dependencies.
- SQL selects, projects, joins, inserts, updates and deletes.

Do this in the practical

- 1 Create the STUDENT table, then alter it by adding a column, and drop a practice table you no longer need.
- 2 Insert, update and delete rows.
- 3 Write SELECT statements that filter with WHERE, compare numbers, and join STUDENT to RESULT.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A primary key:

- A. Identifies one row and does not repeat
- B. Must be a picture
- C. Is always a foreign key of the same table only
- D. Replaces the DBMS

2. Third normal form removes:

- A. Transitive dependencies
- B. All tables
- C. The need for a key
- D. Hardware

3. WHERE in SQL performs:

- A. Selection of rows
- B. Projection only, with no row test
- C. A flowchart
- D. Inheritance

4. A foreign key:

- A. Refers to a primary key, usually of another table
- B. Is a monitor port
- C. Deletes the schema by itself
- D. Is a wireless cell

5. The external level of a database is:

- A. A user's view of part of the data
- B. The physical placement on disk
- C. A C++ destructor
- D. A bus topology

6. INSERT is used to:

- A. Add a row
- B. Draw an entity oval
- C. Boot the operating system
- D. Measure a nibble

7. Many-to-many in a library loan is stored with:

- A. A junction table such as ISSUE
- B. A single cell listing every book
- C. No table at all
- D. A pointer only

8. NOT NULL means:

- A. The column must contain a value
- B. The table is secret
- C. The column is a flowchart
- D. The row is a topology

Answer key

Q	Answer	Why this is the answer
1	A	The primary key uniquely identifies a tuple.
2	A	3NF removes non-key columns that depend on other non-key columns.
3	A	WHERE filters rows. The column list projects columns.
4	A	The foreign key implements a relationship.
5	A	The external level is the view level.
6	A	INSERT is the DML statement that adds data.
7	A	A junction table holds one row per loan and carries both foreign keys.
8	A	NOT NULL is an integrity constraint that rejects an empty value.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Differentiate a flat file and a database.

Model answer

A flat file stores records in one file, and the same fact is often copied onto many rows. A database, managed by a DBMS, stores related tables, reduces that repetition, enforces keys and lets several programs share one controlled copy. A change to a student's name can be made once in the student table.

Q2. Define primary key and foreign key.

Model answer

A primary key is a column or group of columns that identifies one row and cannot be duplicated or left empty. A foreign key is a column that stores the primary key of a row in another table, which creates the relationship. `ISSUE.MemberID` is a foreign key referring to `MEMBER.MemberID`.

Q3. State the rule of each normal form.

Model answer

1NF: each value is atomic and there is no repeating group. 2NF: the table is in 1NF and every non-key column depends on the whole key. 3NF: the table is in 2NF and no non-key column depends on another non-key column.

Q4. Differentiate selection and projection.

Model answer

Selection chooses rows that meet a condition, written with WHERE. Projection chooses columns, written as the list after SELECT. `SELECT StudentName FROM STUDENT WHERE Dept = 'Science'` does both: it projects StudentName and selects the Science rows.

Q5. What is an equijoin?

Model answer

An equijoin combines rows from two tables when related columns are equal, typically a foreign key and a primary key. `STUDENT` joined to `RESULT` on `StudentID` produces one row for each result, carrying the student's name. A cross join has no such condition and pairs every row with every row.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the three levels of abstraction, ER diagrams and normalisation up to 3NF.

- 1 Describe external, conceptual and physical levels.
- 2 Draw or describe entity, attribute and relationship symbols, with BOOK, MEMBER and ISSUES.
- 3 Define 1NF, 2NF and 3NF.
- 4 Walk the student-subject example: partial dependency out at 2NF, HOD out at 3NF.
- 5 Name primary key, foreign key and NOT NULL on the finished tables.

Question 2. Design a library database and write SQL to create a table and to list books on loan.

- 1 State the purpose and the three tables BOOK, MEMBER and ISSUE.
- 2 Give the columns and the primary and foreign keys.
- 3 Justify 3NF: title and member name are not repeated on every loan.
- 4 Write CREATE TABLE for MEMBER or BOOK.
- 5 Write a SELECT with a join that shows member name and book title where ReturnDate is null.

Class XII | Unit 6 | weight 15%

Introduction to multimedia

Board weight about 15% | 20 theory periods and 20 lab periods

Multimedia is the use of more than one medium, such as text, picture, sound and moving image, in one presentation. This lecture names the parts, the file types, the software, and the steps that produce a short video.

What you should be able to do

- Define multimedia and explain its importance.
- Describe text, graphics, animation, audio and video, and name file types for each.
- Distinguish web-based and non-web multimedia applications, and name editing tools.
- Use the ideas of scene, layer and keyframe, and describe export and sharing.

Path through the lecture

- 1 Play a one-minute clip and ask students to name every medium they can see or hear.
- 2 Attach a file extension to each medium before opening the editor.
- 3 Build a 20-second version in the lab first. The curriculum's two-minute video is the full project.

6.1 What multimedia is

Multimedia. The combination of two or more media, such as text, graphics, animation, audio and video, in a single digital presentation.

A textbook page of plain text is one medium. A lesson that shows a labelled diagram, speaks the definition and plays a short clip is multimedia. Colleges use it for lessons, laboratory briefings and presentations. It helps a learner who follows a picture more easily than a paragraph, and it lets one package carry the explanation, the example and the sound.

- A difficult process can be shown as well as described.
- The same lesson can be watched again.
- Text can be searched, while the picture carries the part that is hard to search.
- A presentation can serve a class in the room and a class joining over a network.

6.2 Components and file types

The five components

Component	What it contributes	File types to remember
Text	Titles, labels and definitions	txt, doc, pdf
Graphics	Still pictures and diagrams	jpg, png, gif
Animation	A sequence of drawn or generated frames	gif, and video formats that hold the result
Audio	Speech, music and effects	mp3, wav
Video	Moving pictures, usually with sound	mp4, avi

GIF can be a still picture or a short animation. PNG keeps sharp diagrams and can include transparency. JPEG suits photographs and is lossy: it saves space by discarding some detail. WAV is a full recorded waveform and is large. MP3 is compressed audio. MP4 is the usual container for a finished student video. Use these names in answers even if the lab also produces other types.

6.3 Applications and packages

A **web-based** multimedia application runs in a browser or is delivered as a site: an online lesson, a video page, an interactive diagram. A **non-web** application runs as a program on the computer: a slide show, a video editor, a player. The curriculum names packages such as ProShow Gold for photo and slide video, and JetAudio for playing and working with audio. Current labs often use OpenShot or Shotcut for video and Audacity for sound. Name the curriculum examples, and also name the tool you actually used in the practical.

6.4 Making the video

Scene or canvas. The frame in which you place the parts of the presentation. It is the visible area the viewer will see.

Layer. One level of content stacked with others. A background image can sit on a lower layer and a title on a higher layer, so the title remains editable on its own.

Keyframe. A frame in which you set a property, such as position or size. The software creates the in-between frames, which produces the motion.

- 1 Open a new project and set the canvas to a standard frame such as 1280 by 720.
- 2 Import or record the pictures, the voice and any existing clip. Keep the source files in one project folder.
- 3 Place the background on a lower layer and titles and labels on higher layers.
- 4 Set keyframes where an object should start and where it should finish. Keep motion slow enough to read.
- 5 Record or import narration. Match the length of each sentence to the picture it describes.
- 6 Watch the whole piece from the start and repair a wrong order before you export.
- 7 Export to MP4 for sharing, and keep a second export such as AVI if the question asks for more than one format.
- 8 Upload the finished file to the sharing place your college uses, or to a video site the teacher names, and send the link to the class. Respect the privacy of any person who appears in the clip, and use only pictures and music you have the right to use.

The curriculum project is a video of about two minutes on a topic from this course, for example the OSI layers or the states of a process. Two minutes is long enough to need a plan and short enough to finish in the lab periods.

In the lab

Plan the two minutes on paper first: four scenes of about 30 seconds. Scene 1 states the definition. Scene 2 labels a diagram. Scene 3 gives an example. Scene 4 repeats the definition in one sentence. Then open the editor.

Key terms

Term	Meaning to write in an answer
Multimedia	More than one medium combined in one presentation.
Graphics	Still images.
Animation	Change across frames that creates the sense of motion.
Layer	A separate level of content stacked in a scene.
Keyframe	A frame where a property is set so the software can fill the frames between.
Export	Saving the finished project as a playable file such as MP4.

Bring it together

- Multimedia combines text, graphics, animation, audio and video.
- Learn one file type for each component.
- Web applications are delivered in the browser. Desktop editors such as a video or audio package are non-web applications.
- A scene holds layers. Keyframes set motion. Export, then share with permission to use the material.

Do this in the practical

- 1 Place a title and an imported image on separate layers and save the picture in two formats.
- 2 Import one audio file and one image into one project.
- 3 Add two keyframes that move a label across the canvas.
- 4 Produce a short video, export it as MP4, and show the file to your teacher. Build toward the two-minute piece.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. Multimedia means:

- A. Using more than one medium together
- B. Using only plain text
- C. A type of RAM
- D. A bus topology

2. Which extension is typical of a photograph?

- A. jpg
- B. mp3
- C. txt only
- D. sql

3. A keyframe is:

- A. A frame where you set a property for the animation
- B. A primary key
- C. A keyboard port
- D. An operating system

4. A layer is used so that:

- A. Parts of a scene can be edited separately
- B. RAM is refreshed
- C. A file is binary
- D. A pointer is declared

5. MP3 is:

- A. Compressed audio
- B. A still photograph standard
- C. A C++ keyword
- D. A normal form

6. A web-based multimedia application:

- A. Is delivered through a browser or the web
- B. Cannot contain sound
- C. Is always stored in ROM
- D. Replaces the DBMS

7. Exporting a video means:

- A. Saving the finished presentation as a playable file
- B. Deleting the project
- C. Printing the motherboard
- D. Normalising a table

8. PNG is most suitable in class for:

- A. A sharp diagram
- B. A database key
- C. A process state only
- D. A twisted-pair cable

Answer key

Q	Answer	Why this is the answer
1	A	The word names a combination of media.
2	A	JPEG is a common photographic image format.
3	A	The editor interpolates between keyframes.
4	A	Stacked layers keep the title independent of the background.
5	A	MP3 is an audio format.
6	A	Web-based means the user reaches it through the web.
7	A	Export produces the file you can play or upload.
8	A	PNG keeps crisp edges, which suits diagrams and text in pictures.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Define multimedia and give two advantages.**Model answer**

Multimedia combines two or more media, such as text, sound and video, in one presentation. It can show a process that is hard to follow from text alone, and the learner can play the lesson again. A labelled diagram with a spoken definition is a simple example.

Q2. Name the five components and one file type for each.**Model answer**

Text uses txt or pdf. Graphics use jpg or png. Animation may use gif. Audio uses mp3 or wav. Video uses mp4 or avi. The component is the kind of content. The file type is how that content is stored.

Q3. Differentiate a web-based and a non-web multimedia application.

Model answer

A web-based application is reached through a browser, such as an online lesson with video. A non-web application is a program on the computer, such as a video editor or a desktop player. Both can contain the same five components. They differ in how the user starts them.

Q4. Define scene, layer and keyframe.

Model answer

The scene or canvas is the visible frame. A layer is one stacked level of content, so a title can sit above a picture. A keyframe is a frame where a property such as position is set. The software fills in the frames between keyframes to make movement.

Q5. Why keep source files in one folder before exporting?

Model answer

The project refers to the imported pictures and sounds by their location. If a file is moved, the editor cannot find it and the export is incomplete. One project folder keeps the path stable until the video has been exported.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the components of multimedia and the file types used to store them.

- 1 Define multimedia and give its importance in one short paragraph.
- 2 Explain text, graphics, animation, audio and video, each with a classroom example.
- 3 Attach at least one file type to each, and say when jpg is a better photo format than png is for a diagram, or keep the comparison inside the graphics row.
- 4 Distinguish a web delivery and a desktop package, naming one package.

Question 2. Describe how you would make a two-minute educational video and share it.

- 1 Plan four short scenes on a syllabus topic.
- 2 Set the canvas, import media, and arrange layers.
- 3 Use keyframes for one simple movement and add narration.
- 4 Export to MP4 and name a second format.
- 5 Upload or share the file with the teacher's method, and use only material you have permission to use.

Class XII | Unit 7 | weight 10%

Wireless and mobile communication

Board weight about 10% | 15 theory periods

Wireless communication carries data without a cable between the devices. Mobile communication is wireless communication that continues while the user moves. This lecture compares the short-range technologies, the satellite orbits, and the cellular network that a phone actually uses.

What you should be able to do

- Define wireless and mobile communication and the basic radio terms.
- Compare Wi-Fi, WiMAX, Bluetooth, infrared and microwave on range, speed and frequency.
- Describe long-distance wireless links and the GEO, MEO and LEO orbits.
- Explain cells, base stations, frequency reuse, MSC and BTS.
- Describe the generations of mobile networks and distinguish a fixed ad hoc network from a MANET.

Path through the lecture

- 1 Separate wireless from mobile with two examples: a fixed microwave link, and a phone in a moving bus.
- 2 Put the short-range table on one board. It is a favourite short question.
- 3 Draw three hexagons, a BTS in each, and one MSC. Then attach the generations to that drawing.

7.1 Terms

Wireless communication. Communication that uses electromagnetic waves rather than a cable between the sender and the receiver.

Mobile communication. Wireless communication that allows the user to move and to keep the connection, typically through a cellular network.

- The **frequency spectrum** is the range of frequencies available for radio signals. Different services are given different bands so they do not all talk on top of each other.
- A **radio signal** is the electromagnetic wave that carries the data.
- A **radio transceiver** both transmits and receives. A phone contains one. The name joins transmitter and receiver.
- An **access point** is the device that joins wireless users to a wired network. A home or lab Wi-Fi box is an access point, often combined with a router.
- **Line of sight** means the path between two antennas is clear. Microwave links need it. A hill or a building in the path interrupts them.

7.2 Short-range technologies

The figures below are the textbook comparisons used for this unit. A particular product can differ. In an answer, the order matters more than a single brand's specification: Bluetooth is personal, Wi-Fi is a room or a building, WiMAX was designed for a city, infrared is a short beam, and microwave can jump between distant towers when the path is clear.

Short-distance wireless technologies

Technology	Typical range	Speed in this comparison	Frequency band
Bluetooth	About 10 metres, more for some classes	Lower than Wi-Fi, enough for headsets and small file transfers	2.4 GHz
Infrared	About a metre to a few metres, and only in a clear beam	Low, suitable for a remote control or a short link	Infrared light, above radio frequencies
Wi-Fi	Tens of metres indoors, more in the open	High, from many megabits per second upward	2.4 GHz and 5 GHz
Microwave	Kilometres between aligned antennas	High on a point-to-point link	Microwave bands, and the path must be line of sight
WiMAX	A metropolitan area, up to tens of kilometres in the design	Broadband, lower or comparable to later Wi-Fi in practice	Licensed bands around 2 to 11 GHz in the original designs

7.3 Long distance and satellites

Long-distance wireless communication crosses cities, countries or oceans by radio towers, microwave chains or satellites. A satellite receives a signal on one frequency, the uplink, and sends it back toward Earth on another, the downlink. The height of the orbit decides the delay, the coverage and the number of satellites needed.

Orbits used with satellite communication and navigation

Orbit	Height and behaviour	What to remember
GEO, geostationary	About 36,000 km above the equator. The orbital period matches the Earth's rotation, so the satellite stays over the same region.	One satellite covers a large fixed area. The signal has a noticeable delay. Weather and television links use this idea.
MEO, medium Earth orbit	Roughly 2,000 to 20,000 km.	Navigation constellations such as GPS use MEO, around 20,000 km. Coverage of the globe needs a group of satellites, fewer than a low orbit needs.
LEO, low Earth orbit	Below about 2,000 km.	The delay is smaller and the satellite moves quickly across the sky. Continuous coverage needs many satellites. The curriculum's phrase 'types of GPS' is answered with these three orbits, and you should add that the GPS constellation itself flies in MEO.

7.4 The cellular network

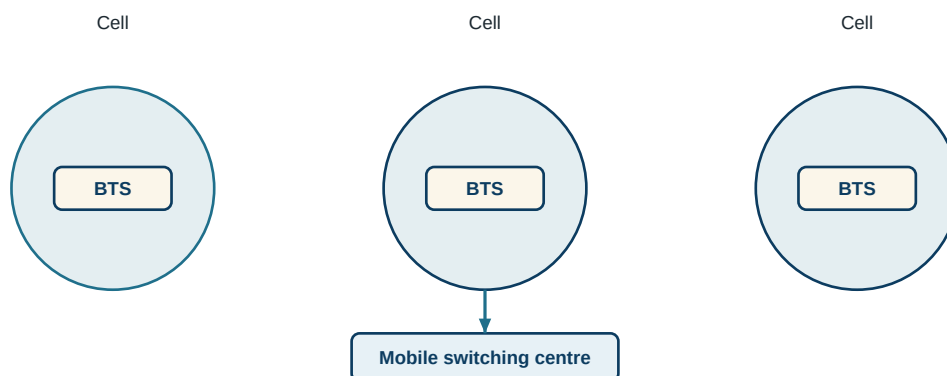


Figure 7.1 Cells, a base transceiver in each, and a mobile switching centre.

- A **cell** is the area served by one base station. Diagrams draw it as a hexagon so the cells tile a map.
- A **base transceiver station (BTS)** is the radio equipment and antenna in that cell. It is the part the phone talks to.

- A **base station** is the broader name for the equipment that serves a cell, including the BTS and its controller.
- **Frequency reuse** means the same channel frequencies can be used again in cells that are far enough apart. Neighbouring cells use different frequencies so their calls do not collide. That reuse is why a limited spectrum can serve a whole city.
- The **mobile switching centre (MSC)** connects cells to each other and to the ordinary telephone network. It tracks which cell a phone is in and supports the handover when a caller crosses a boundary.
- The **interface between cells** is the handover: the call is passed from one base station to the next while the conversation continues.

7.5 Generations

Each generation is a family of technologies for the cellular network. The 2019 curriculum pairs the names as 2G (GSM), 3G (GPRS), 4G (LTE) and 5G. Write those pairs if the question uses them. For a precise answer, add the engineering names: GSM is 2G voice and digital service, GPRS is the packet-data step often called 2.5G, 3G is the UMTS family, 4G is LTE, and 5G is the following generation. The curriculum's bracket '5G (Future)' reflects the year of the curriculum. 5G networks have since been deployed in many countries. What you should say about 5G is that it is designed for higher data rates and lower delay than LTE, and for a much larger number of connected devices.

Generations

Name in the question	What the user gained
2G GSM	Digital mobile calls and text messages.
GPRS, listed with 3G in the curriculum note	Packet data over the mobile network, so the phone could carry internet traffic. Technically this step is 2.5G, and 3G then raised the data rate further.
4G LTE	Much higher data rates for video and ordinary internet use on the phone.
5G	A further rise in rate and a fall in delay, and support for many more devices.

7.6 Ad hoc networks

Ad hoc network. A network formed by devices that communicate directly and cooperate to forward data, without depending on fixed infrastructure such as an access point or a base station.

A **fixed ad hoc network** is set up for a time among nodes that do not move, for example laptops in a hall that forward for one another. A **mobile ad hoc network (MANET)** is an ad hoc network whose nodes move. Routes change as devices come and go, so the network has to rediscover paths. A disaster team whose radios form a network without a tower is the example to give.

Exam point

Keep three comparisons ready: Wi-Fi with Bluetooth, GEO with LEO, and a cellular network with a MANET. Each is a 3-mark table.

Key terms

Term	Meaning to write in an answer
Wireless	Communication by electromagnetic waves, without a data cable between the ends.
Access point	A device that connects wireless stations to a network.

Term	Meaning to write in an answer
Line of sight	A clear path between antennas.
Cell	The geographic area served by one base station.
BTS	The radio transmitter and receiver that serves a cell.
MSC	The switch that connects cells and handles handover.
GEO	A geostationary orbit, fixed relative to the ground.
MANET	A mobile ad hoc network, with moving nodes and no fixed infrastructure.

Bring it together

- Wireless removes the cable. Mobile communication also survives movement.
- Bluetooth, infrared, Wi-Fi, microwave and WiMAX differ in range, speed and band.
- GEO stays over one region at great height. GPS uses MEO. LEO needs many fast-moving satellites.
- A cellular network reuses frequencies across cells and hands a call over at the boundary through the MSC.
- GSM, GPRS, LTE and 5G are the generation story. An ad hoc network has no fixed base station. A MANET also has moving nodes.

Do this in the practical

- 1 From the lab Wi-Fi, identify the access point name and compare its range with a Bluetooth headset in the same room.
- 2 Draw a three-cell sketch of the college area and mark where a handover would happen if a caller walked between them.

Check yourself

Attempt every item before you read the key. Section A of the Computer Science paper is a set of multiple-choice questions.

1. A transceiver:

- A. Transmits and receives
- B. Only prints
- C. Stores a primary key
- D. Compiles C++

2. Infrared communication typically requires:

- A. A short, clear beam
- B. A geostationary satellite
- C. A coaxial backbone across a city
- D. An MSC

3. Wi-Fi is described by the family:

- A. IEEE 802.11
- B. ASCII
- C. 1NF
- D. BIOS

4. A geostationary satellite:

- A. Stays over the same region of the Earth
- B. Orbits inside the classroom
- C. Is a type of RAM
- D. Replaces the SIM card

5. GPS satellites operate in:

- A. Medium Earth orbit
- B. The CMOS battery
- C. A register of the CPU
- D. A text file

6. Frequency reuse in cellular networks allows:

- A. The same frequencies to be used again in distant cells
- B. One frequency for the entire world with no planning
- C. Removal of the need for a phone
- D. Text files to become binary

7. The MSC is responsible for:

- A. Switching calls and supporting handover between cells
- B. Erasing EPROM
- C. Compiling a class
- D. Drawing a flowchart oval

8. A MANET differs from a fixed ad hoc network because:

- A. Its nodes move
- B. It uses only fibre
- C. It is a normal form
- D. It cannot forward data

Answer key

Q	Answer	Why this is the answer
1	A	The device combines a transmitter and a receiver.
2	A	Infrared is short range and does not pass through walls.
3	A	IEEE 802.11 is the wireless LAN standard behind Wi-Fi.
4	A	Its period matches the Earth's rotation.
5	A	The GPS constellation uses MEO.
6	A	Cells far apart can reuse channels, which multiplies capacity.
7	A	The mobile switching centre connects the cells and the wider telephone network.
8	A	MANET means a mobile ad hoc network.

Short answers

A 3-mark answer needs a definition and two clear points, or a three-row difference. Write in sentences.

Q1. Differentiate wireless communication and mobile communication.

Model answer

Wireless communication sends data by electromagnetic waves without a cable between the ends. A fixed microwave link is wireless and the ends stay still. Mobile communication is wireless communication that continues while the user moves, as in a cellular phone call. Mobile communication is therefore a moving form of wireless communication.

Q2. Compare Wi-Fi and Bluetooth on range and use.

Model answer

Bluetooth is a short personal link, about 10 metres, used for a headset or a small accessory, in the 2.4 GHz band. Wi-Fi covers a room or a building, carries much more data, and joins devices to a local network through an access point, using 2.4 GHz or 5 GHz. Wi-Fi is the network. Bluetooth is the short personal connection.

Q3. Differentiate GEO and LEO.

Model answer

A GEO satellite orbits at about 36,000 km and stays over one region, so a fixed dish can use it, at the cost of a longer delay. A LEO satellite orbits below about 2,000 km, moves quickly across the sky and has a shorter delay. Continuous service from LEO needs a large constellation.

Q4. What is a cell, and why do neighbouring cells use different frequencies?

Model answer

A cell is the area covered by one base station. Neighbouring cells use different frequencies so that calls near the boundary do not interfere. The same frequency can be reused in a cell that is farther away. That is frequency reuse.

Q5. Define an ad hoc network and a MANET.

Model answer

An ad hoc network is formed by devices that talk directly and forward data for one another, without a fixed access point or base station. A MANET is an ad hoc network whose nodes move, so the routes have to be found again as the topology changes.

Detailed answers

Section C asks for a connected answer. Open with a definition, develop the points below, and close with one example.

Question 1. Explain the parts of a cellular network and the idea of frequency reuse and handover.

- 1 Define wireless and mobile communication.
- 2 Define cell, BTS, base station and MSC, and describe how a phone reaches the wider network.
- 3 Explain frequency reuse with neighbouring cells on different channels.
- 4 Explain handover as the caller crosses a cell boundary.
- 5 Add one sentence on 2G, 4G and 5G so the network has a technology as well as a map.

Question 2. Compare short-distance wireless technologies and the three satellite orbits.

- 1 Give the comparison table for Bluetooth, infrared, Wi-Fi, microwave and WiMAX: range, speed and frequency.
- 2 Define line of sight and attach it to microwave and infrared.
- 3 Describe GEO, MEO and LEO with height and one use.
- 4 State that GPS flies in MEO.

- 5 Close with the access point as the device that joins a short-range Wi-Fi user to the wired network.